



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 4

Tương tranh và đồng bộ

Nội dung chính - Chương 4



Mục lục

- 4.1. Các khái niệm cơ bản
- 1.2. Độc quyền truy xuất – Giải pháp phần mềm
- 1.3. Đồng bộ hóa – Giải pháp phần cứng
- 1.4. Giải pháp đồng bộ cơ bản
- 1.5. Các bài toán kinh điển và kỹ thuật nâng cao

Từ khóa

Concurrency

Race condition

Critical section

Mutual exclusion

Mutex

Semaphore

Monitor

Spinlock

Deadlock

Starvation

Trọng tâm kiến thức



Nhận biết vấn đề

Giải thích được vì sao nhiều tiến trình cùng truy cập dữ liệu có thể gây sai kết quả.

Hiểu cách bảo vệ

Mô tả vùng tới hạn, độc quyền truy xuất và các yêu cầu của một giải pháp đúng.

Sử dụng công cụ

Chọn được mutex, semaphore, monitor hoặc message passing cho tình huống phù hợp.

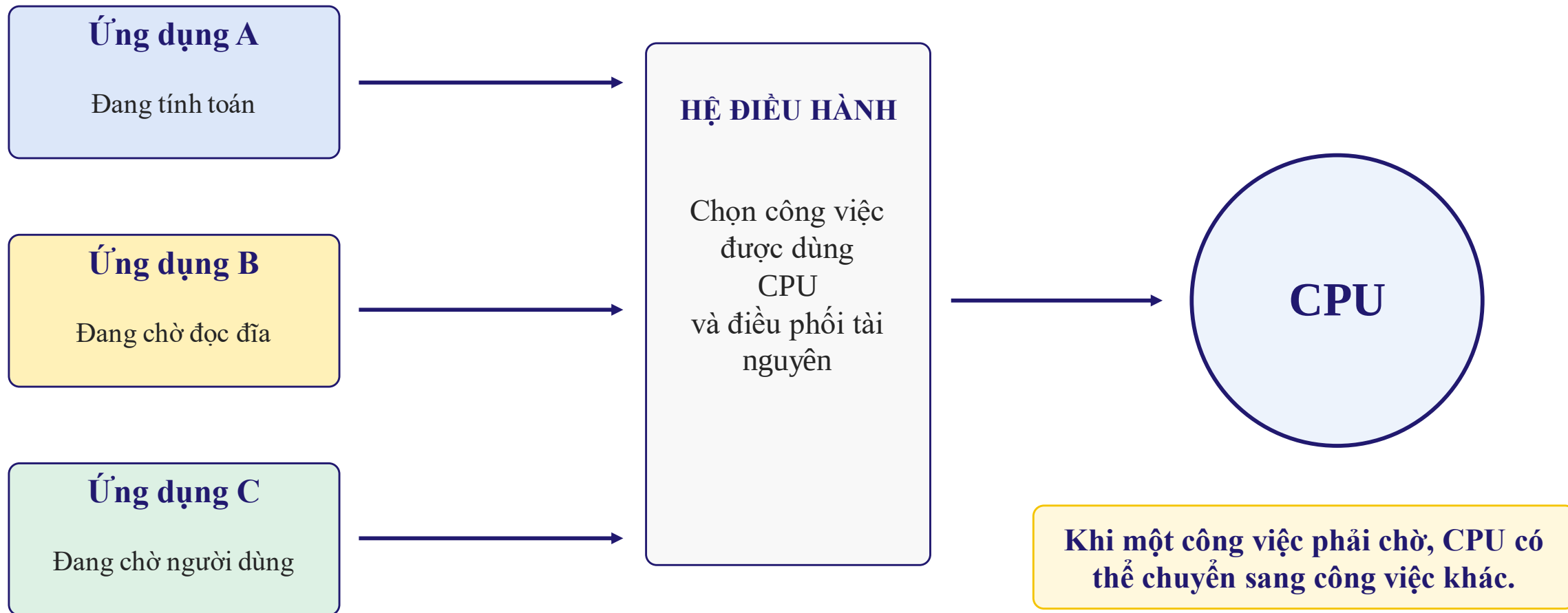
Câu hỏi định hướng: Làm thế nào để “cùng chạy” mà không “giẫm chân” lên nhau?



4.1. Các khái niệm cơ bản

Bắt đầu từ những tình huống quen thuộc: nhiều công việc cùng dùng một tài nguyên.

Vì sao cần cho nhiều công việc cùng chạy?



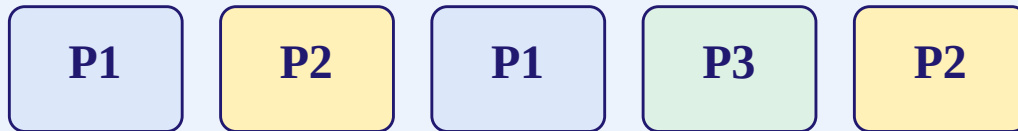


Tương tranh và song song khác nhau thế nào?

TƯƠNG TRANH (Concurrency)

Một CPU chuyển đổi nhanh giữa nhiều công việc.

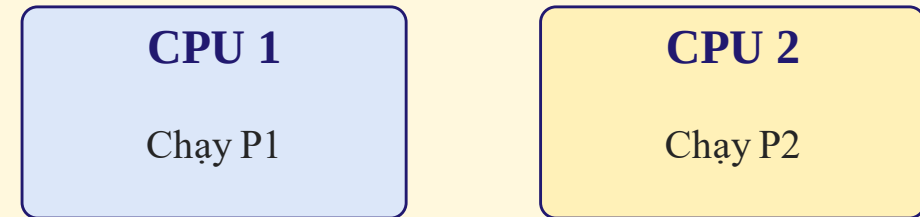
Thời gian →



Có cảm giác cùng chạy, nhưng tại một thời điểm CPU chỉ xử lý một công việc.

SONG SONG (Parallelism)

Nhiều CPU hoặc nhiều lõi thực hiện thật sự cùng lúc.



Hai công việc được xử lý ở cùng một thời điểm.

Cả hai trường hợp đều cần đồng bộ khi có dữ liệu dùng chung.

Tài nguyên dùng chung là gì?



Biến trong bộ nhớ

count, balance, queue

Tập dữ liệu

file log, bảng điểm

Thiết bị

máy in, ổ đĩa

Cấu trúc hệ điều hành

bảng tiến trình, bộ đệm

Khi nhiều tiến trình cùng đọc và ghi một tài nguyên, thứ tự thực hiện có thể làm thay đổi kết quả.



Điều gì có thể xảy ra khi không đồng bộ?



1

Mất cập nhật

Hai tiến trình cùng sửa một giá trị, nhưng một kết quả bị ghi đè.

2

Dữ liệu chưa nhất quán

Một tiến trình đọc dữ liệu khi tiến trình khác mới sửa được một phần.

3

Kết quả thay đổi theo lần chạy

Cùng một chương trình nhưng mỗi lần có thể cho kết quả khác.

Thuật ngữ: Điều kiện tranh chấp (race condition).

Kết quả phụ thuộc vào việc ai chạy trước, ai chạy sau.



Ví dụ: Hai tiến trình cùng tăng biến count

Giả sử count ban đầu bằng 5 và cả hai tiến trình đều thực hiện $count = count + 1$.

Mong đợi

5 → 6 → 7

Một cách thực hiện có thể xảy ra

Bước	Tiến trình A	Tiến trình B	count trong bộ nhớ
1	Đọc 5		5
2		Đọc 5	5
3	Tính 6		5
4		Tính 6	5
5	Ghi 6		6
6		Ghi 6	6

Kết quả cuối cùng là 6, không phải 7 → mất một lần cập nhật.



Vì sao `count++` không phải là một bước duy nhất?

Ở mức chương trình, ta thấy:

```
count = count + 1;
```

Nhưng CPU thường thực hiện thành nhiều bước:

- 1 Đọc `count` từ bộ nhớ
- 2 Tăng giá trị trong thanh ghi
- 3 Ghi kết quả trở lại bộ nhớ

Nếu tiến trình khác chen vào giữa các bước, kết quả có thể sai.



Vùng tới hạn (Critical Section)

Vùng tới hạn là đoạn mã truy cập hoặc thay đổi tài nguyên dùng chung.

Ví dụ mã

```
// Phần không dùng dữ liệu chung  
chuẩn_bị();  
  
// VÙNG TỚI HẠN  
count = count + 1;  
  
// Phần còn lại  
hiển_thị();
```

Tiến trình A

Muốn vào

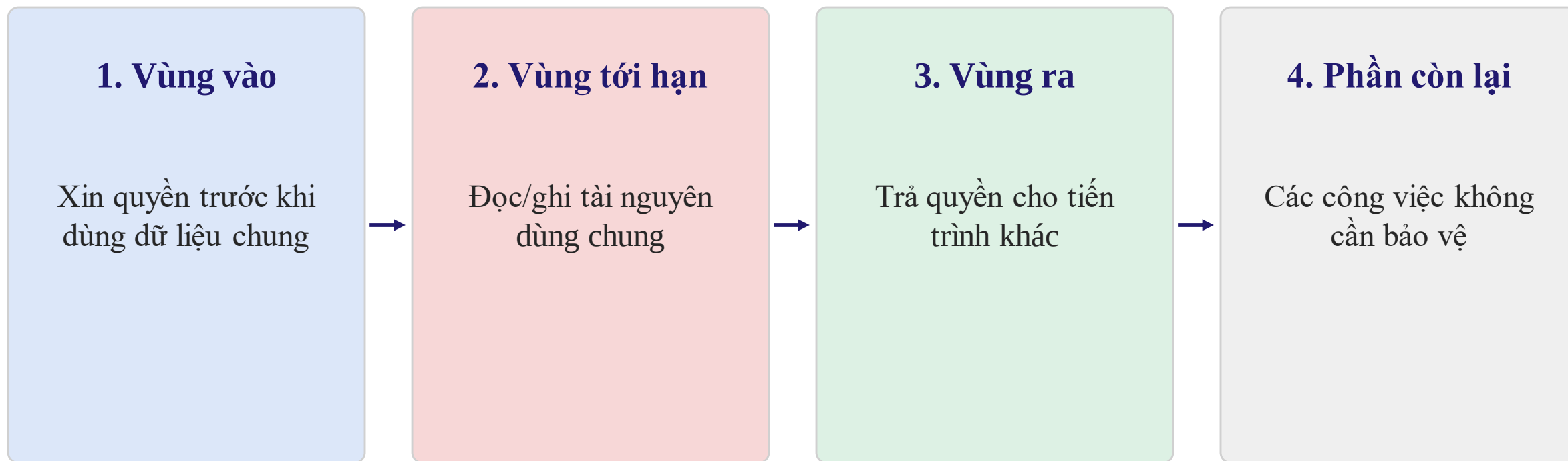
Tiến trình B

Muốn vào

VÙNG TỚI HẠN

Tại một thời điểm chỉ một tiến trình được vào.

Một tiến trình thường có bốn phần liên quan



Đồng bộ chủ yếu tập trung vào vùng vào và vùng ra.



Một giải pháp đúng cần đáp ứng điều gì?

- 1 Độc quyền truy xuất** Không có hai tiến trình cùng ở trong vùng tới hạn.
- 2 Không cản trở vô lý** Nếu vùng tới hạn đang trống, tiến trình có nhu cầu phải có cơ hội vào.
- 3 Không chờ mãi** Mỗi tiến trình phải được vào sau một số lần chờ hữu hạn.

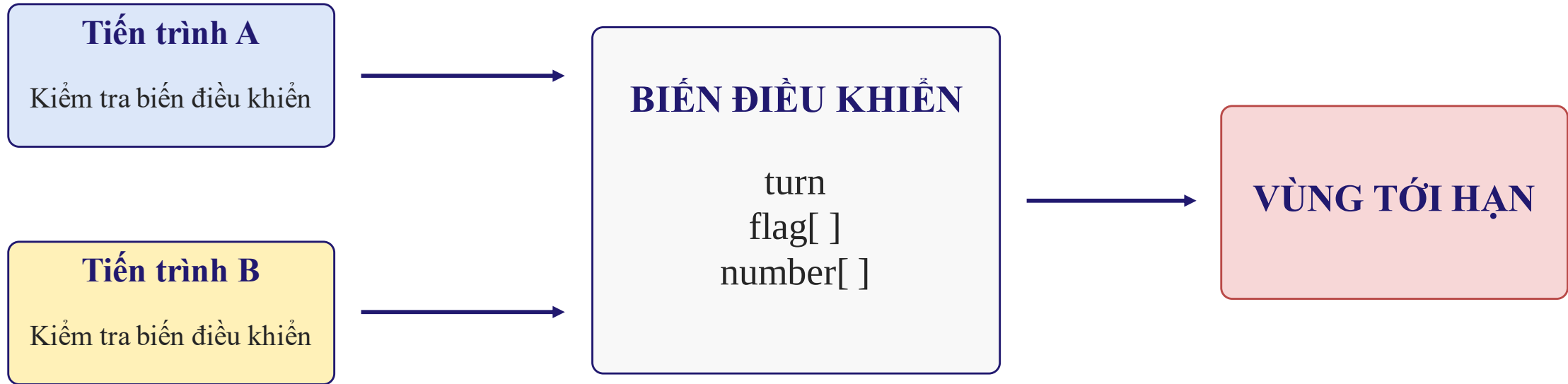
Ba yêu cầu này thường được gọi là: Mutual exclusion, progress, bounded waiting.



4.2. Độc quyền truy xuất – Giải pháp phần mềm

Các tiến trình tự phối hợp bằng biến dùng chung, không cần lệnh phần cứng đặc biệt.

Ý tưởng chung của giải pháp phần mềm

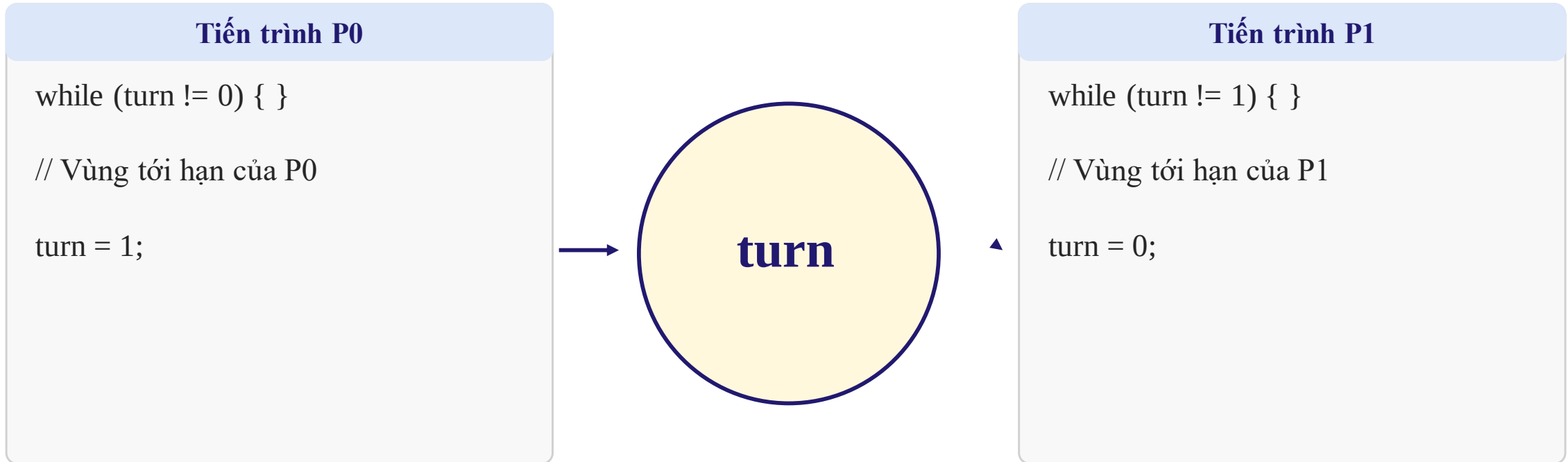


Điểm khó: Phải bảo đảm đúng trong mọi cách xen kẽ giữa các tiến trình.



Giải pháp luân phiên nghiêm ngặt

Dùng biến turn để quy định ai được vào vùng tới hạn.



Ưu điểm: dễ hiểu và bảo đảm không vào cùng lúc.

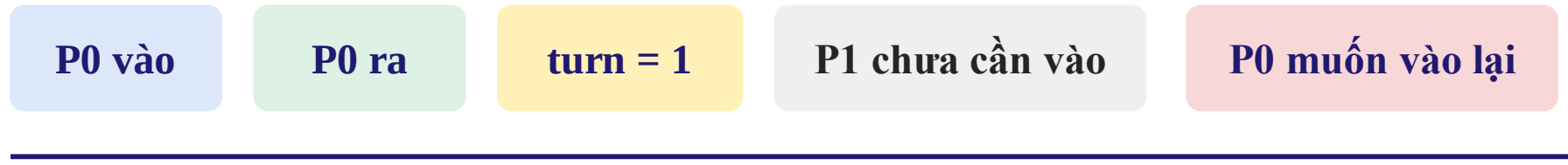
Nhược điểm: bắt buộc phải thay phiên, kể cả khi bên kia không cần vào.



Vì sao luân phiên nghiêm ngặt chưa tốt?

Tình huống: P0 vừa ra khỏi vùng tới hạn, đặt $turn = 1$. Nhưng P1 đang làm việc khác.

Thời gian →



P0 phải chờ P1, dù vùng tới hạn đang trống.

Giải pháp này vi phạm yêu cầu “Không cản trở vô lý”.

Giải pháp Peterson cho hai tiến trình



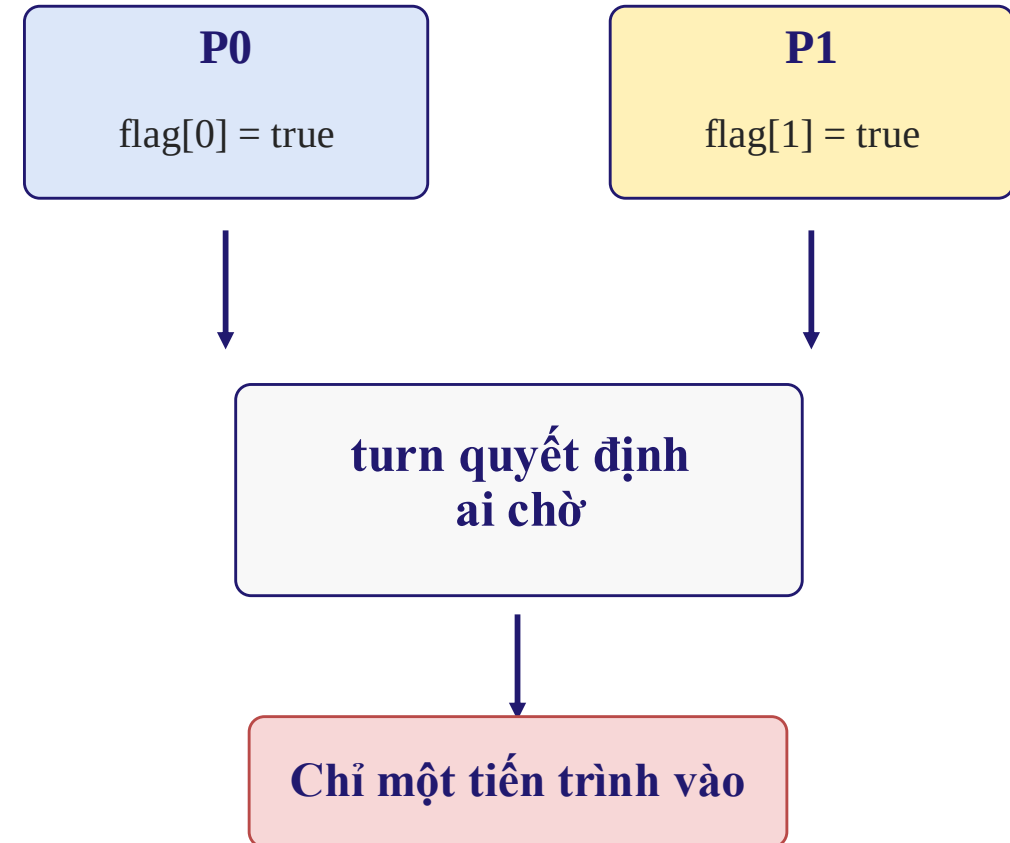
Peterson kết hợp hai ý tưởng:

flag[i]

Tiến trình i thông báo: “Tôi muốn vào”.

turn

Nếu cả hai cùng muốn vào, nhường lượt theo biến turn.





Mã giả Peterson

Mỗi tiến trình i có tiến trình còn lại là j.

Tiến trình i

```
flag[i] = true;
turn = j;

while (flag[j] && turn == j) {
    // chờ
}

// VÙNG TỐI HẠN

flag[i] = false;
```

- 1 Báo rằng mình muốn vào
- 2 Nhường ưu tiên cho tiến trình kia
- 3 Chờ nếu tiến trình kia cũng muốn vào và đang được ưu tiên
- 4 Ra khỏi vùng tới hạn thì hạ cờ

Áp dụng cho hai tiến trình và giả định việc đọc/ghi biến điều khiển là nguyên tử.

Nếu P0 và P1 cùng muốn vào thì sao?



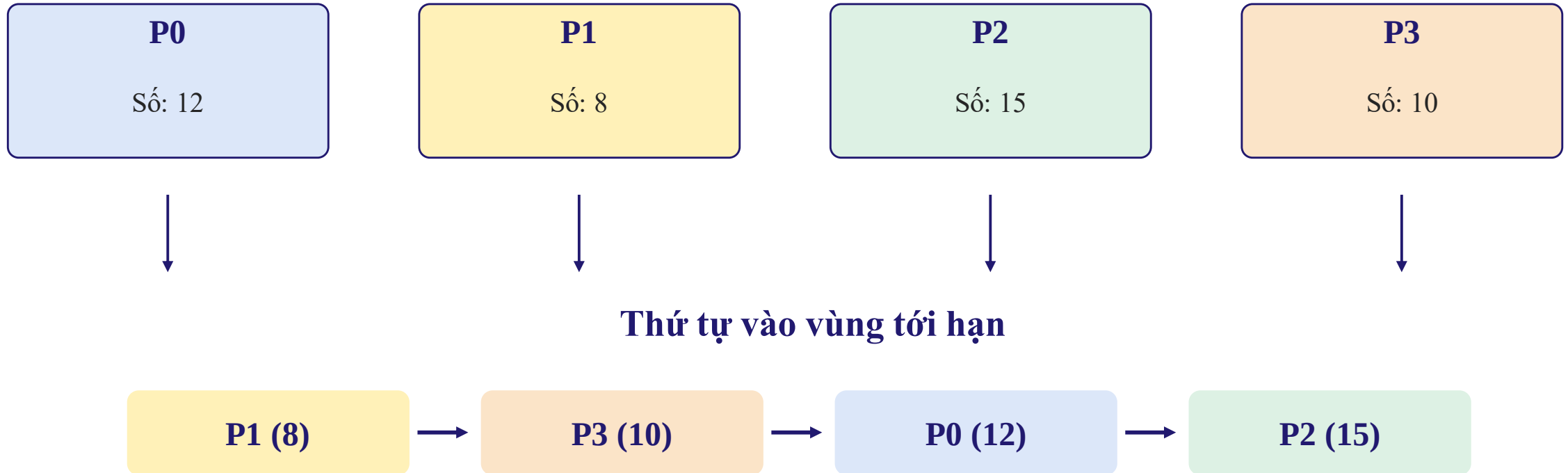
Thời điểm	P0	P1	Kết quả
1	flag[0] = true	flag[1] = true	Cả hai cùng xin vào
2	turn = 1	turn = 0	Lệnh ghi cuối cùng quyết định turn
3	Kiểm tra điều kiện	Kiểm tra điều kiện	Một bên chờ, một bên vào
4	Hạ flag khi ra	Tiếp tục kiểm tra	Bên còn lại được vào

Điểm quan trọng: Không có trường hợp cả hai cùng vượt qua vòng chờ.



Thuật toán “lấy số thứ tự” cho nhiều tiến trình

Ý tưởng giống quầy phục vụ: ai lấy số nhỏ hơn được phục vụ trước.



Hữu ích để hiểu nguyên lý, nhưng trong thực tế thường dùng mutex hoặc semaphore do hệ điều hành cung cấp.

So sánh các giải pháp phần mềm



Giải pháp	Số tiến trình	Ưu điểm	Hạn chế
Luân phiên nghiêm ngặt	2	Rất dễ hiểu	Bắt buộc thay phiên, có thể chờ vô lý
Peterson	2	Đáp ứng ba yêu cầu cơ bản	Chỉ phù hợp để học nguyên lý; có chờ bận
Lấy số thứ tự	Nhiều	Công bằng theo thứ tự	Cài đặt phức tạp hơn

Bài học chính: dùng biến dùng chung vẫn có thể tạo độc quyền truy xuất, nhưng rất dễ sai khi cài đặt.



4.3. Đồng bộ hóa – Giải pháp phần cứng

CPU cung cấp các lệnh đặc biệt giúp việc kiểm tra và cập nhật khóa diễn ra trong một bước.



Vì sao cần hỗ trợ từ phần cứng?

Khó khăn của giải pháp phần mềm:

- Đọc và ghi biến điều khiển có thể bị xen kẽ.
- Mã phức tạp, khó kiểm tra mọi trường hợp.
- Dễ dùng sai khi số tiến trình tăng.



LỆNH NGUYÊN TỬ CỦA CPU

Test-and-Set

Compare-and-Swap

Kiểm tra và thay đổi khóa
trong một thao tác không bị chen ngang.

Phần cứng làm “viên gạch”, hệ điều hành xây mutex, semaphore và các khóa khác.

Tắt ngắt: Cách đơn giản trên hệ một CPU



Ý tưởng

```
disable_interrupts();
```

```
// Vùng tới hạn
```

```
enable_interrupts();
```

Khi tắt ngắt

CPU không chuyển sang tiến trình khác

Khi bật lại

Hệ thống tiếp tục xử lý ngắt bình thường

Không phù hợp cho chương trình người dùng: tắt ngắt quá lâu làm máy chậm phản hồi.

Trên hệ nhiều CPU, tắt ngắt ở một CPU không ngăn CPU khác truy cập dữ liệu.



Lệnh Test-and-Set

Lệnh này đọc giá trị cũ và đặt khóa thành true trong cùng một thao tác.

Mô tả đơn giản

```
boolean TestAndSet(boolean *lock) {  
    old = *lock;  
    *lock = true;  
    return old;  
}
```

Cách dùng để tạo khóa:

```
while (TestAndSet(&lock)) { }  
  
// Vùng tới hạn  
lock = false;
```

lock = false



TestAndSet



lock = true

Tiến trình đầu tiên nhận old = false nên được vào.
Các tiến trình sau nhận true và tiếp tục chờ.



Spinlock: Khóa bằng cách “quay vòng” chờ



Phù hợp khi thời gian giữ khóa rất ngắn; không phù hợp nếu phải chờ lâu.

- Ưu: không cần chuyển trạng thái tiến trình.
- Nhược: tốn thời gian CPU khi chờ.

**while (lock)
{ kiểm tra lại }**

CPU vẫn bận kiểm tra khóa liên tục



Lệnh Compare-and-Swap (CAS)

CAS chỉ thay đổi giá trị nếu giá trị hiện tại đúng như mong đợi.

Ý tưởng

```
CAS(address, expected, newValue)
```

```
if (*address == expected) {  
    *address = newValue;  
    return true;  
}  
return false;
```

Giá trị hiện tại

=

Giá trị mong đợi



Đúng → ghi giá trị mới

Sai → không thay đổi

CAS là nền tảng của nhiều cấu trúc dữ liệu không khóa (lock-free).

So sánh các hỗ trợ phần cứng



Cách làm	Phù hợp	Ưu điểm	Hạn chế
Tắt ngắt	Nhân HĐH, một CPU	Rất đơn giản	Không dùng cho người dùng; kém hiệu quả trên nhiều CPU
Test-and-Set	Khóa đơn giản	Dễ tạo spinlock	Chờ bận; có thể thiếu công bằng
Compare-and-Swap	Khóa và cấu trúc nâng cao	Linh hoạt, phổ biến	Khó thiết kế đúng; vẫn có thể lặp nhiều lần

Hỗ trợ phần cứng không tự giải quyết mọi vấn đề; hệ điều hành vẫn cần xây cơ chế chờ, đánh thức và công bằng.



Chờ bận và chờ ngủ: Chọn cách nào?

CHỜ BẬN (Busy Waiting)

Tiến trình kiểm tra khóa liên tục.

- Không tốn chi phí chuyển trạng thái.
- Tồn CPU trong thời gian chờ.
- Hợp với khóa giữ rất ngắn.

CHỜ NGỦ (Blocking)

Tiến trình bị đưa vào hàng đợi và nhường CPU.

- Không lãng phí CPU khi phải chờ lâu.
- Có chi phí ngủ/đánh thức.
- Hợp với vùng tới hạn dài hơn.

Quy tắc dễ nhớ: Chờ rất ngắn → spin; chờ có thể lâu → block.

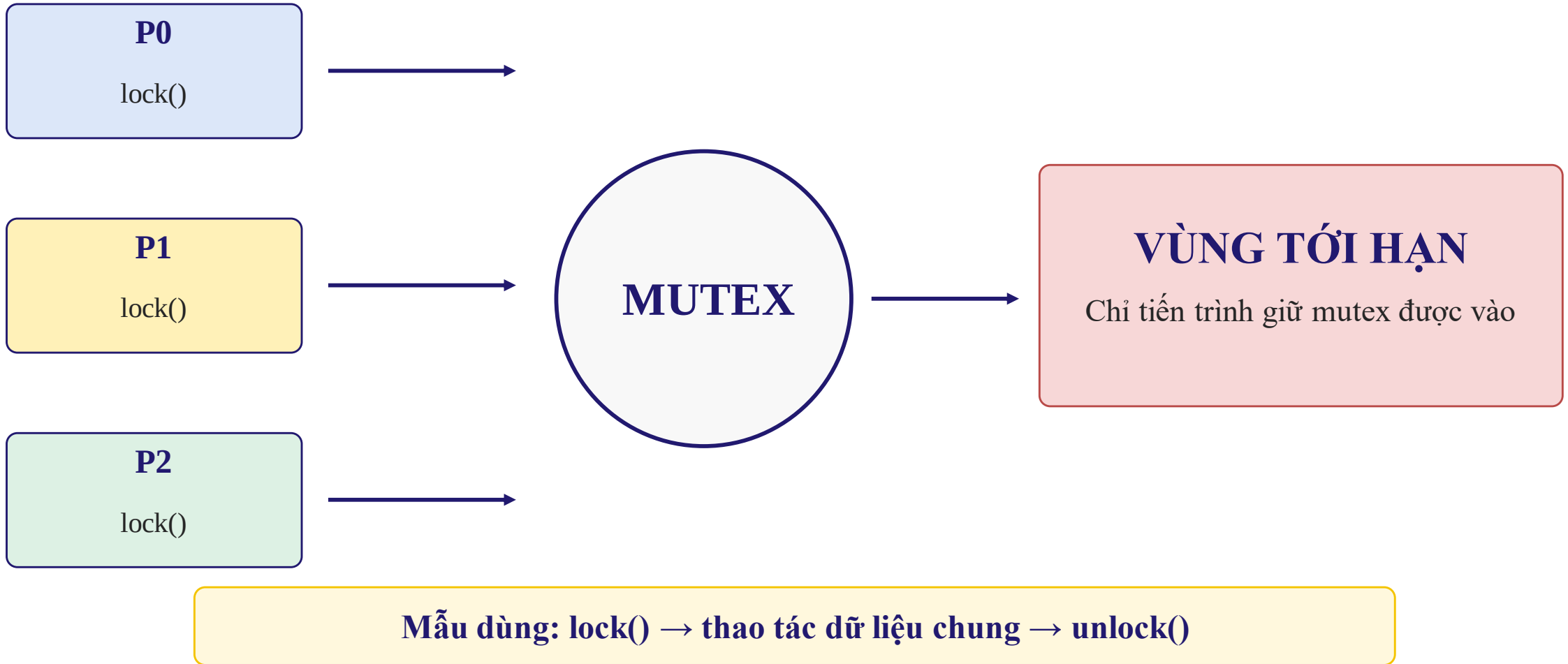


4.4. Các giải pháp đồng bộ cơ bản

Những công cụ mà lập trình viên và hệ điều hành thường dùng trong thực tế.



Mutex: Khóa chỉ cho một tiến trình đi qua



Semaphore: Biển đếm dùng để điều phối



Semaphore lưu một số nguyên và chỉ thay đổi qua hai thao tác nguyên tử:

wait(S)

Xin một “vé”.
Nếu không còn vé, tiến
trình phải chờ.

signal(S)

Trả một “vé”.
Có thể đánh thức một
tiến trình đang chờ.

Semaphore nhị phân

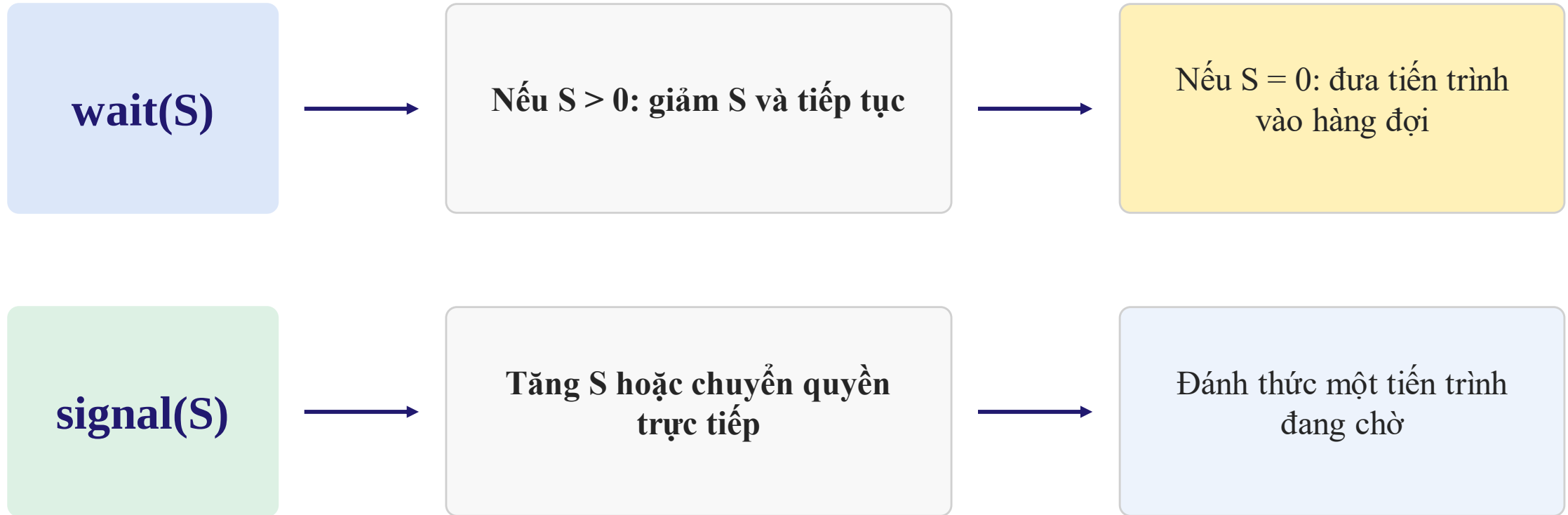
Giá trị 0 hoặc 1 – gần giống
mutex

Semaphore đếm

Giá trị từ 0 trở lên – quản lý
nhiều tài nguyên giống nhau



wait() và signal() hoạt động thế nào?



Hai thao tác phải nguyên tử: Không được để tiến trình khác chen vào giữa khi đang cập nhật S .

Dùng semaphore nhị phân để bảo vệ biến count



Mã giả

```
semaphore mutex = 1;  
  
wait(mutex);  
count = count + 1;  
signal(mutex);
```

S = 1

Khóa đang rảnh



wait(S)

Một tiến trình lấy khóa



S = 0

Tiến trình khác phải chờ



signal(S)

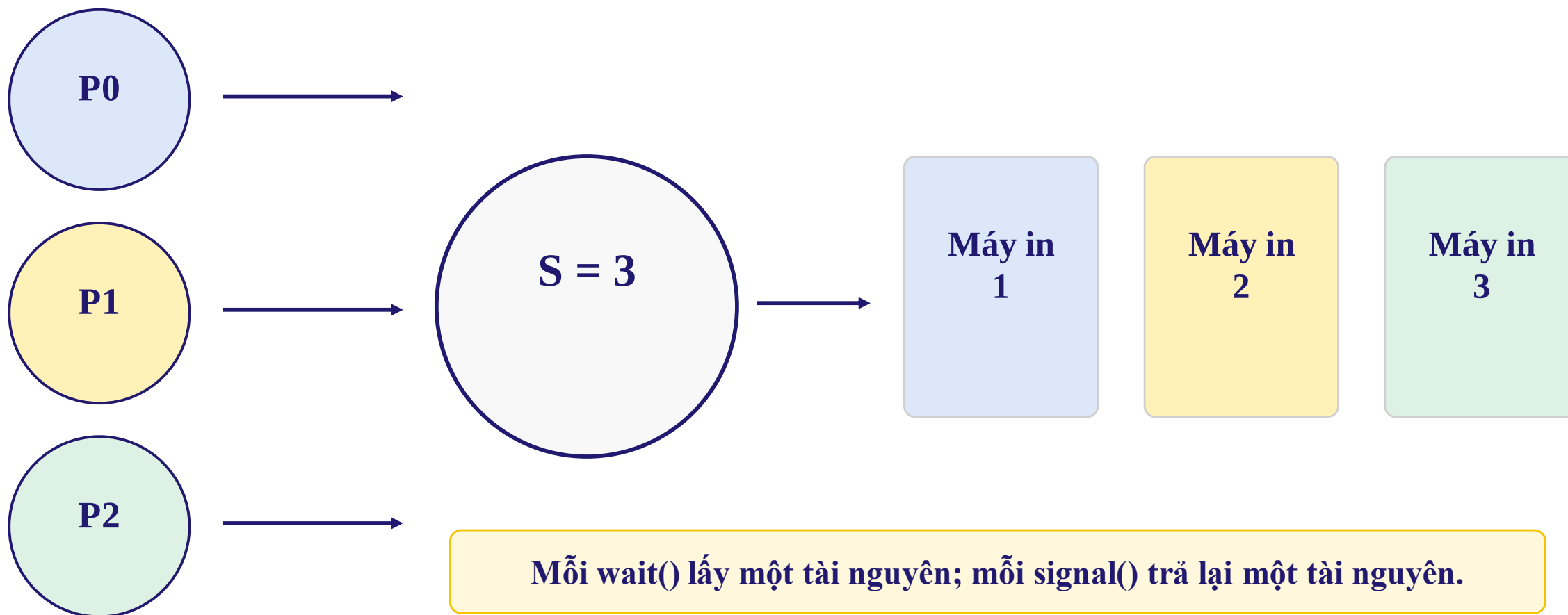
Trả khóa và đánh thức

Luôn đặt signal() ở mọi đường ra khỏi vùng tới hạn, kể cả khi có lỗi.

Semaphore đếm: Quản lý nhiều tài nguyên giống nhau

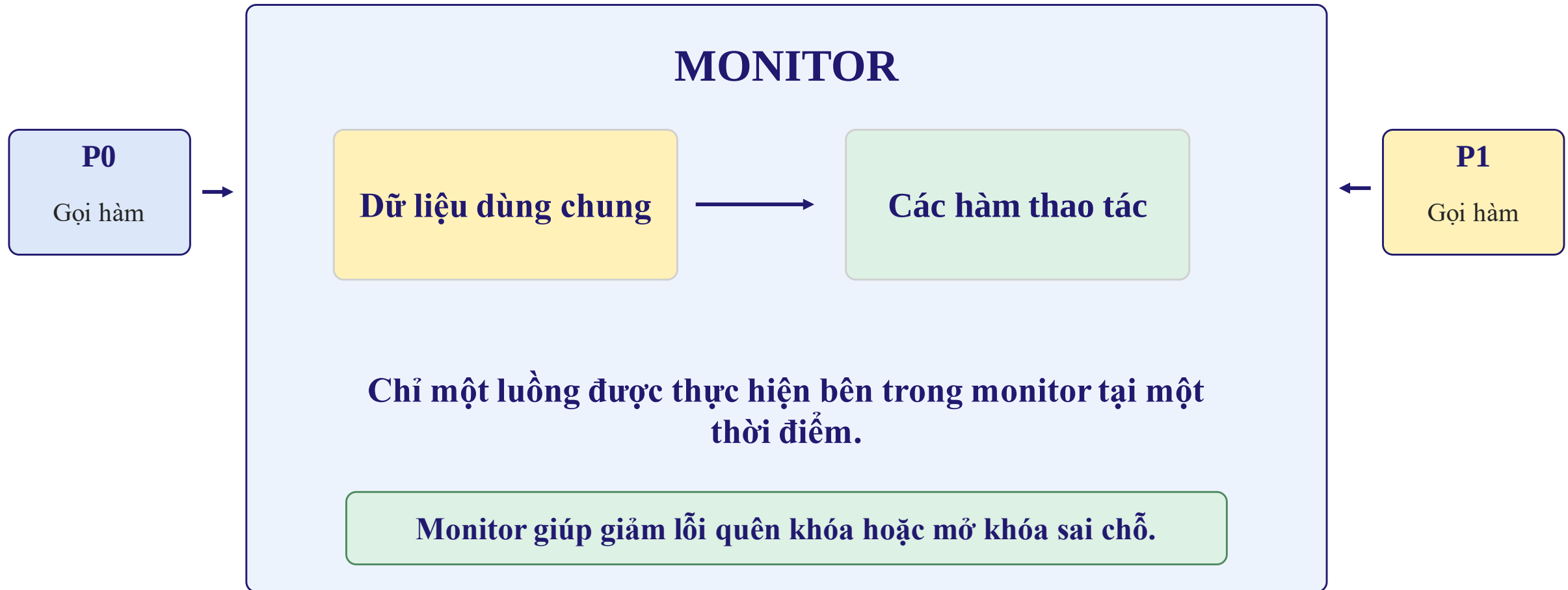


Ví dụ có 3 máy in dùng chung.





Monitor: Gom dữ liệu và thao tác vào một “phòng an toàn”





Biến điều kiện: Chờ đúng lúc, đánh thức đúng việc

Dùng bên trong monitor khi một luồng phải chờ một điều kiện trở thành đúng.

- 1 **Kiểm tra điều kiện** → buffer còn trống không?
- 2 **wait(condition)** → Nếu chưa đúng: tạm ngủ và nhường monitor
- 3 **signal(condition)** → Luồng khác thay đổi dữ liệu và đánh thức

wait trên biến điều kiện khác wait trên semaphore: nó dùng để chờ một điều kiện logic.

Truyền thông điệp: Không chia sẻ dữ liệu trực tiếp



Ưu điểm

Giảm chia sẻ trực tiếp; phù hợp tiến trình ở máy khác nhau.

Hạn chế

Có chi phí sao chép và quản lý thông điệp.



Chọn công cụ đồng bộ nào?

Bảo vệ một dữ liệu dùng chung



Mutex

Quản lý N tài nguyên giống nhau



Semaphore đếm

Chờ một điều kiện trong cấu trúc dữ liệu



Monitor + biến điều kiện

Các tiến trình không muốn chia sẻ bộ nhớ



Truyền thông điệp

Khóa giữ rất ngắn trong nhân HĐH



Spinlock

Không có công cụ “tốt nhất cho mọi trường hợp”. Chọn theo thời gian chờ, số tài nguyên và cách giao tiếp.



4.5. Các bài toán đồng bộ kinh điển và kỹ thuật nâng cao

Luyện cách nhận diện tài nguyên, điều kiện chờ và mục tiêu công bằng.

Bài toán Producer – Consumer



Producer phải chờ khi buffer đầy

Consumer phải chờ khi buffer rỗng

Giải pháp Producer – Consumer bằng semaphore



mutex = 1

Bảo vệ buffer

empty = N

Số ô trống

full = 0

Số ô đã có dữ liệu

Producer

```
wait(empty);  
wait(mutex);
```

đưa dữ liệu vào buffer;

```
signal(mutex);  
signal(full);
```

Consumer

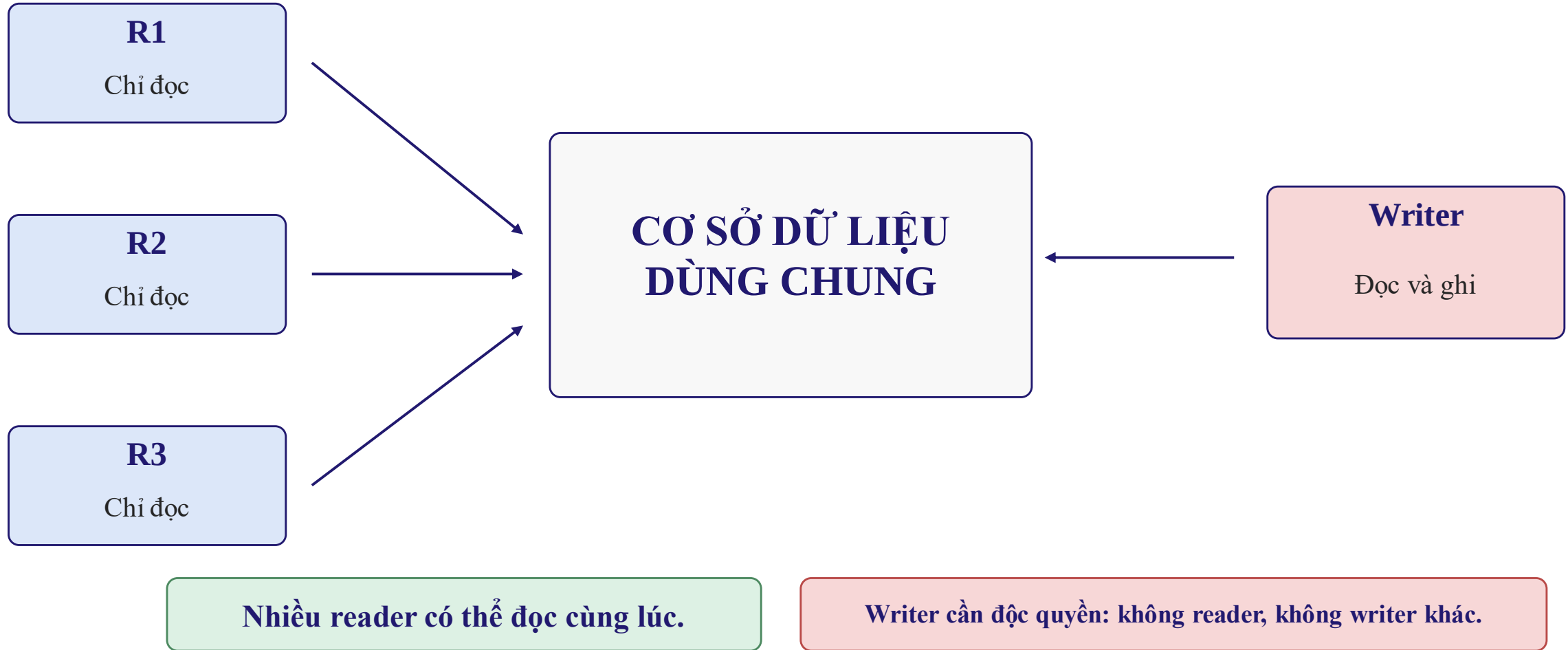
```
wait(full);  
wait(mutex);
```

lấy dữ liệu khỏi buffer;

```
signal(mutex);  
signal(empty);
```

Thứ tự wait rất quan trọng để tránh khóa chéo.

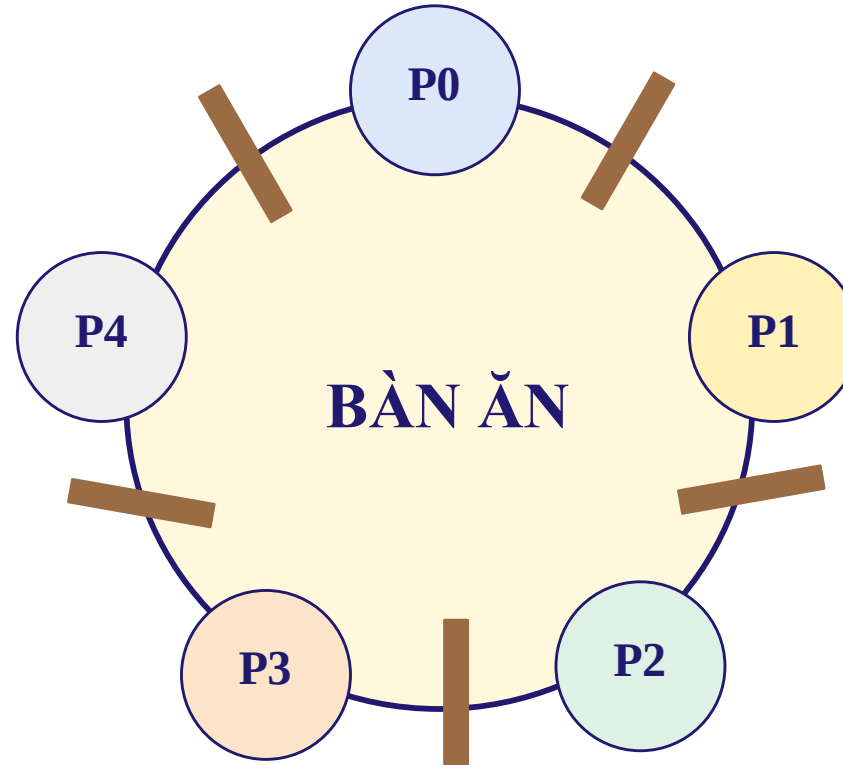
Bài toán Readers – Writers



Bài toán năm triết gia dùng đũa



Mỗi người cần hai chiếc đũa để ăn. Mỗi chiếc đũa được dùng chung với người bên cạnh.



Nếu ai cũng cầm đũa bên trái rồi chờ đũa bên phải → tất cả chờ nhau.



Các cách tránh bế tắc trong bài toán triết gia

1 Giới hạn số người

Chỉ cho tối đa 4 người cùng có lấy đĩa.

2 Lấy theo thứ tự

Đánh số đĩa và luôn lấy chiếc số nhỏ trước.

3 Một người lấy ngược

Bốn người lấy trái trước, một người lấy phải trước.

4 Dùng người phục vụ

Chỉ được lấy đĩa khi “phục vụ” cho phép.

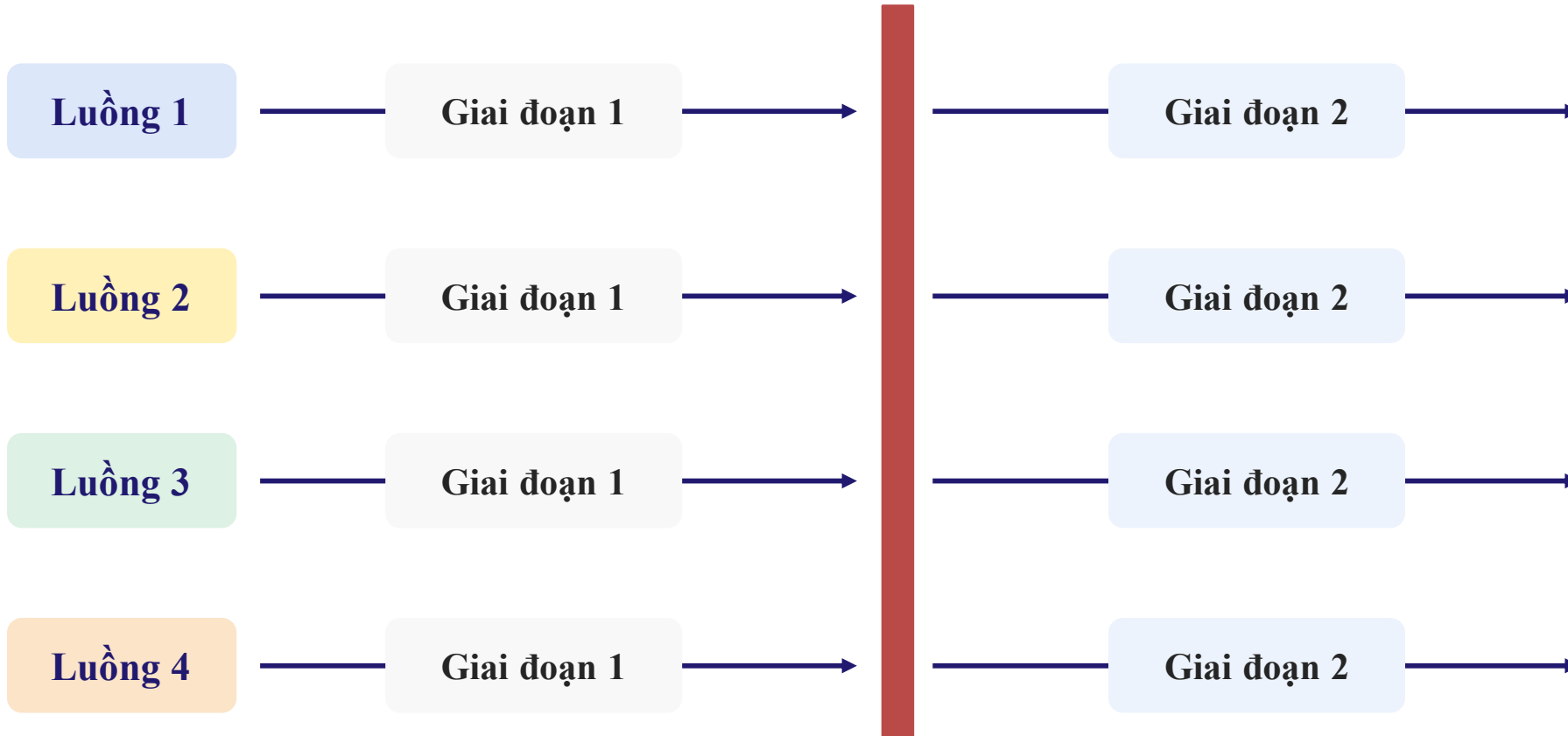
Điểm chung: Phá vỡ vòng chờ lẫn nhau.



Barrier: Tất cả cùng đến rồi mới đi tiếp

Barrier dùng khi một công việc được chia thành nhiều giai đoạn song song.

BARRIER



Luồng đến sớm phải chờ đến khi tất cả luồng còn lại cũng đến barrier.

Phân biệt bế tắc, đói tài nguyên và livelock



Bế tắc (Deadlock)

Các tiến trình chờ nhau mãi;
không ai tiếp tục được.

Ví dụ

P0 giữ A chờ B; P1 giữ B chờ A.

Đói tài nguyên (Starvation)

Một tiến trình liên tục bị tiến
trình khác vượt lên.

Ví dụ

Reader mới đến liên tục khiến
writer không được chạy.

Livelock

Các tiến trình vẫn thay đổi
trạng thái nhưng không tiến gần
đến kết quả.

Ví dụ

Hai bên liên tục nhường nhau.

Cả ba đều làm hệ thống kém hiệu quả, nhưng nguyên nhân và cách xử lý khác nhau.

Nguyên tắc giúp tránh lỗi đồng bộ



1

Giữ vùng tới hạn ngắn nhất có thể.

2

Luôn mở khóa ở mọi đường thoát, kể cả khi có lỗi.

3

Dùng cùng một thứ tự khi lấy nhiều khóa.

4

Không gọi hàm chậm hoặc I/O khi đang giữ khóa nếu có thể tránh.

5

Ghi rõ dữ liệu nào được bảo vệ bởi khóa nào.

6

Ưu tiên công cụ mức cao thay vì tự cài khóa thấp cấp.

Đồng bộ đúng trước, tối ưu sau.



Ví dụ tổng hợp: Cập nhật số dư an toàn

Trước khi đồng bộ

```
// Chưa an toàn  
void deposit(int amount) {  
    balance = balance + amount;  
}
```



Sau khi dùng mutex

```
mutex.lock();  
  
balance = balance + amount;  
  
mutex.unlock();
```

KẾT QUẢ

Mỗi lần cập nhật
được thực hiện
trọn vẹn

Phân tích

- ✓ Tài nguyên dùng chung: balance.
- ✓ Vùng tới hạn: thao tác đọc – cộng – ghi balance.
- ✓ Mutex bảo đảm chỉ một luồng cập nhật tại một thời điểm.
- ✓ Cần dùng finally/defer/RAII để chắc chắn mở khóa khi có lỗi.

Tổng kết Chương 4



Nhiều tiến trình cùng tiến triển và có thể dùng chung tài nguyên
Đoạn mã cần bảo vệ để tránh kết quả phụ thuộc vào thứ tự chạy.
Mutex, semaphore, monitor, biến điều kiện và message passing.
Producer–Consumer, Readers–Writers, Dining Philosophers
Deadlock, starvation, livelock và priority inversion



Câu hỏi thảo luận

Vì sao thêm nhiều khóa không phải lúc nào cũng làm chương trình an toàn hơn?

Khi nào chờ bận có thể nhanh hơn chờ ngủ?

Một hệ thống ngân hàng nên ưu tiên tính đúng hay hiệu năng? Vì sao?

Trong thực tế, bạn đã gặp ứng dụng nào bị “đơ” do chờ tài nguyên?

Email: minhnl@utc.edu.vn